

Relatório Técnico: Sistemas Embarcados

Identificação do Aluno	Número USP	Data de Entrega
Giovanni Antonio Silva de Oliveira	14575064	xx/xx/202x

1. Identificação do Experimento

- **Título:** Atividade 6 - Threads
- **Disciplina:** PSI3441 - Arquitetura de Sistemas Embarcados
- **Professor:** Gustavo Pamplona Rehder

2. Respostas, Comentários e Análises

A atividade foi desenvolvida criando-se libs para uso em arquivos para o ADC(`adc.h/c`), botão(`digitalIn.h/c`) e sensor(`acelerometro.h/c`), e no `main.c` é feita a criação das threads e as atividades dela em si.

como observação do último tópico (7) foram testados os seguintes casos, com o resultado logo a frente(sabendo que sempre se mantinha a ordem de 2 prints do ADC, e 1 do sensor):

- **ADC e Sensor com mesma prioridade:** Foi observado que, ao manter a mesma prioridade, na mudança de modo de ADC para completo, a primeira mensagem (thread) que enviava o dado para o serial variava entre o adc e o sensor, possivelmente ocorrendo devido a ambas começarem no mesmo instante, sendo atraso interno do temporizador ditando qual iria começar naquele momento (portanto aleatório da nossa perspectiva).
- **ADC com prioridade maior:** Agora, a primeira mensagem que é mandada sempre é a do ADC, ficando a do sensor como segundo ou terceiro print no terminal.

- **Sensor com prioridade maior:** O mesmo do caso anterior, apenas que ao invés do ADC, o acelerômetro começa como primeira medição na tela.

É interessante comentar apenas que as prioridades são ditas com base em parâmetros de pré-compilação no início do arquivo main.c

3. Código-Fonte

```
////////////////////////////////////
////////////////////////////////////Main.c
////////////////////////////////////

#include <zephyr/kernel.h>
#include "adc.h"
#include "acelerometro.h"
#include "digitalIn.h"

// Tamanho de memória para cada thread
#define STACK_SIZE 1024

// prioridade da thread
#define ADC_THREAD_PRIORITY 5
#define ACC_THREAD_PRIORITY 7

// define teste
#define MODO_TESTE
#define TES_THREAD_PRIORITY 8

// para alternar os modos do botao
static volatile bool modo_botao = false;

//////////////////////////////////// Callback do
botao////////////////////////////////////
void botao_pressionado(void)
{
    // Alterna o estado do modo
    modo_botao = !modo_botao;

    if (modo_botao) {
        printk("\n---> MUDOU PARA MODO ADC + Acelerometro\n\n");
    } else {
```

```

        printk("\n--> MUDOU PARA MODO apenas ADC\n\n");
    }
}

////////// thread
adc//////////
void thread_adc_fn(void *arg1, void *arg2, void *arg3)
{
    for(;;)
    {
        // independentemente do modo
        uint32_t milivolts = adc_read_mv();
        printk("[ADC] Valor lido: %d mV\n", milivolts);

        // Dorme por 500 ms
        k_msleep(500);
    }
}

////////// thread
accel//////////
void thread_accel_fn(void *arg1, void *arg2, void *arg3)
{
    accel_data_t dados_accel;

    for(;;)
    {
        // Só tenta ler e imprimir se estiver no Modo Completo
        if (modo_botao) {
            if (accel_read(&dados_accel)) {
                printk("[ACCEL] X: %.2f | Y: %.2f | Z: %.2f\n",
                    (double)dados_accel.x, (double)dados_accel.y,
                    (double)dados_accel.z);
            } else {
                printk("[ACCEL] Falha na leitura!\n");
            }
        }

        // Dorme por 1000 ms
        k_msleep(1000);
    }
}

```

```

//////////////////////////////////// thread teste
////////////////////////////////////
#ifdef MODO_TESTE
void thread_teste_fn(void *arg1, void *arg2, void *arg3)
{
    for(;;)
    {
        k_msleep(10000);

        // Alterna o modo
        modo_botao = !modo_botao;

        if (modo_botao) {
            printk("\n[TESTE AUTO - 10s] ---> MODO COMPLETO
ATIVADO\n\n");
        } else {
            printk("\n[TESTE AUTO - 10s] ---> MODO ADC ATIVADO\n\n");
        }
    }
}

K_THREAD_DEFINE(
    teste_tid,
    STACK_SIZE,
    thread_teste_fn,
    NULL,
    NULL,
    NULL,
    TES_THREAD_PRIORITY,
    0,
    0
);

#endif

//K_THREAD_DEFINE(
//    nome_da_thread,        // Um nome interno para a thread
//    tamanho_da_pilha,      // Memória reservada (geralmente 1024 bytes)
//    funcao_da_thread,      // O nome da função que tem o loop da thread
//    arg1, arg2, arg3,      // Argumentos (geralmente NULL)
//    prioridade,            // Número da prioridade (menor = mais
importante)
//    opcoes,                // 0 por padrão

```

```

//      delay_inicial          // 0 para iniciar na hora
//);

// declaratacao thread adc
K_THREAD_DEFINE(
    adc_tid,
    STACK_SIZE,
    thread_adc_fn,
    NULL,
    NULL,
    NULL,
    ADC_THREAD_PRIORITY,
    0,
    0
);

// declaratacao thread accel
K_THREAD_DEFINE(
    accel_tid,
    STACK_SIZE,
    thread_accel_fn,
    NULL,
    NULL,
    NULL,
    ACC_THREAD_PRIORITY,
    0,
    0
);

int main(void)
{
    printk("=== Inicializando Sistema ===\n");

    // 1. Inicializa o ADC
    if (!adc_init()) {
        printk("Erro ao iniciar ADC.\n");
    }

    // 2. Inicializa Acelerômetro
    if (!accel_init()) {
        printk("Erro ao iniciar Acelerometro.\n");
    }
}

```

```
// 3. Inicializa Botão com a função de Callback
if (!meu_botao_init(botao_pressionado)) {
    printk("Erro ao iniciar Botao.\n");
}

printk("=== Sistema Rodando! Pressione o botão para trocar de modo
===\n\n");

// O Main entra em repouso absoluto. As threads assumem o controle.
for(;;) k_sleep(K_FOREVER);

return 0;
}
```

4. Referências e Links Externos

Utilize este campo para referenciar repositórios de controle de versão, vídeos demonstrativos ou documentos complementares.

- **Link do Repositório (GitHub/GitLab):**

<https://github.com/Djovanne/PSI3441---Arquitetura-de-Sistemas-Embarcados/tree/main/Atividade%206%20-%20Threads>